SIAM J. SCI. COMPUT.
Vol. 34, No. 4, pp. C170–C191
© 2012 Society for Industrial and Applied Mathematics

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

PARALLEL SPARSE MATRIX-MATRIX MULTIPLICATION AND INDEXING: IMPLEMENTATION AND EXPERIMENTS*

AYDIN BULUÇ† AND JOHN R. GILBERT‡

Abstract. Generalized sparse matrix-matrix multiplication (or SpGEMM) is a key primitive for many high performance graph algorithms as well as for some linear solvers, such as algebraic multigrid. Here we show that SpGEMM also yields efficient algorithms for general sparse-matrix indexing in distributed memory, provided that the underlying SpGEMM implementation is sufficiently flexible and scalable. We demonstrate that our parallel SpGEMM methods, which use two-dimensional block data distributions with serial hypersparse kernels, are indeed highly flexible, scalable, and memory-efficient in the general case. This algorithm is the first to yield increasing speedup on an unbounded number of processors; our experiments show scaling up to thousands of processors in a variety of test scenarios.

Key words. parallel computing, numerical linear algebra, sparse matrix-matrix multiplication, SpGEMM, sparse matrix indexing, sparse matrix assignment, two-dimensional data decomposition, hypersparsity, graph algorithms, sparse SUMMA, subgraph extraction, graph contraction, graph batch update

AMS subject classifications. 05C50, 05C85, 65F50, 68W10

DOI. 10.1137/110848244

1. Introduction.

We describe scalable parallel implementations of two sparse matrix kernels. The first, generalized sparse matrix-matrix multiplication (SpGEMM), computes the product of two sparse matrices over a general semiring. The second, SpRef, performs generalized indexing into a sparse matrix: Given vectors I and J of row and column indices, SpRef extracts the submatrix $\mathbf{A}(\mathsf{I}, \mathsf{J})$. Our novel approach to SpRef uses SpGEMM as its key subroutine, which regularizes the computation and data access patterns; conversely, applying SpGEMM to SpRef emphasizes the importance of an SpGEMM implementation that handles arbitrary matrix shapes and sparsity patterns, and a complexity analysis that applies to the general case.

Our main contributions in this paper are as follows: first, we show that SpGEMM leads to a simple and efficient implementation of SpRef; second, we describe a distributed-memory implementation of SpGEMM that is more general in application and more flexible in processor layout than before; and, third, we report on extensive experiments with the performance of SpGEMM and SpRef. We also describe an algorithm for sparse matrix assignment (SpAsgn) and report its parallel performance. The SpAsgn operation, formally $\mathbf{A}(\mathsf{I}, \mathsf{J}) = \mathbf{B}$, assigns a sparse matrix to a submatrix of another sparse matrix. It can be used to perform streaming batch updates to a graph.

*Submitted to the journal's Software and High-Performance Computing section September 16, 2011; accepted for publication (in revised form) April 27, 2012; published electronically July 26, 2012. This work was supported in part by NSF grant CNS-0709385 and by grants from Intel Corporation and Microsoft Corporation. The U.S. Government retains a nonexclusive, royalty-free license to publish or reproduce the published form of this contribution, or allow others to do so, for U.S. Government purposes. Copyright is owned by SIAM to the extent not limited by these rights.

http://www.siam.org/journals/sisc/34-4/84824.html

†Computational Research Division, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, CA 94720 (abuluc@lbl.gov). This author's work was supported by the ASCR Office in the DOE Office of Science under contract DE-AC02-05CH11231.

‡Computer Science Department, University of California, Santa Barbara, CA 93106-5110 (gilbert@cs.ucsb.edu).

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Parallel algorithms for SpGEMM and SpRef, as well as their theoretical performance, are described in sections 3 and 4. We present the general SpGEMM algorithm and its parallel complexity before SpRef since the latter uses SpGEMM as a subroutine and its analysis uses results from the SpGEMM analysis. Section 3.1 summarizes our earlier results on the complexity of various SpGEMM algorithms on distributed memory. Section 3.3 presents our algorithm of choice, Sparse SUMMA, in a more formal way than before, including a pseudocode general enough to handle different blocking parameters, rectangular matrices, and rectangular processor grids. The reader interested only in parallel SpRef can skip these sections and go directly to section 4, where we describe our SpRef algorithm, its novel parallelization, and its analysis. Section 5 gives an extensive performance evaluation of these two primitives using large scale parallel experiments, including a performance comparison with similar primitives from the Trilinos package. Various implementation decisions and their effects on performance are also detailed.

2. Notation. Let $\mathbf{A} \in \mathbb{S}^{m \times n}$ be a sparse rectangular matrix of elements from a semiring $\mathbb{S}$. We use $nnz(\mathbf{A})$ to denote the number of nonzero elements in $\mathbf{A}$. When the matrix is clear from the context, we drop the parentheses and simply use nnz. For sparse matrix indexing, we use the convenient MATLAB colon notation, where $\mathbf{A}(:, i)$ denotes the ith column, $\mathbf{A}(i, :)$ denotes the ith row, and $\mathbf{A}(i, j)$ denotes the element at the (i, j)th position of matrix $\mathbf{A}$. Array and vector indices are 1-based throughout this paper. The length of an array I, denoted by $len(\mathsf{I})$, is equal to its number of elements. For one-dimensional (1D) arrays, $\mathsf{I}(i)$ denotes the ith component of the array. We use flops($\mathbf{A} \cdot \mathbf{B}$), pronounced "flops," to denote the number of nonzero arithmetic operations required when computing the product of matrices $\mathbf{A}$ and $\mathbf{B}$. Since the flops required to form the matrix triple product differ depending on the order of multiplication, flops$((\mathbf{AB}) \cdot \mathbf{C})$ and flops$(\mathbf{A} \cdot (\mathbf{BC}))$ mean different things. The former is the flops needed to multiply the product $\mathbf{AB}$ with $\mathbf{C}$, whereas the latter is the flops needed to multiply $\mathbf{A}$ with the product $\mathbf{BC}$. When the operation and the operands are clear from context, we simply use flops. The MATLAB `sparse(i,j,v,m,n)` function, which is used in some of the pseudocode, creates an $m \times n$ sparse matrix $\mathbf{A}$ with nonzeros $\mathbf{A}(i(k), j(k)) = v(k)$.

In our analyses of parallel running time, the latency of sending a message over the communication interconnect is α, and the inverse bandwidth is β, both expressed in terms of time for a floating-point operation (also accounting for the cost of cache misses and memory indirections associated with that floating point operation). $f(x) = \Theta(g(x))$ means that f is bounded asymptotically by g both above and below.

3. Sparse matrix-matrix multiplication. SpGEMM is a building block for many high-performance graph algorithms, including graph contraction [25], breadth-first search from multiple source vertices [10], peer pressure clustering [34], recursive all-pairs shortest-paths [19], matching [33], and cycle detection [38]. It is a subroutine in more traditional scientific computing applications such as multigrid interpolation and restriction [5] and Schur complement methods in hybrid linear solvers [37]. It also has applications in general computing, including parsing context-free languages [32] and colored intersection searching [29].

The classical serial SpGEMM algorithm for general sparse matrices was first described by Gustavson [26] and was subsequently used in MATLAB [24] and CSparse [20]. That algorithm, shown in Figure 3.1, runs in $O(\text{flops} + nnz + n)$ time, which is optimal for flops $\geq \max\{nnz, n\}$. It uses the popular compressed sparse column (CSC) format for representing its sparse matrices. Algorithm 1 gives the pseudocode for this columnwise serial algorithm for SpGEMM.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

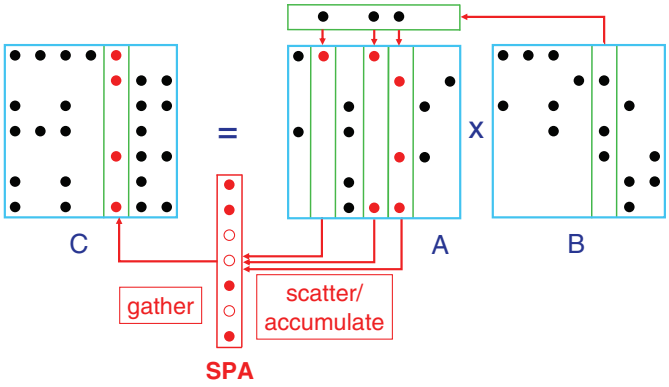


FIG. 3.1. *Multiplication of sparse matrices stored by columns* [11]. *Columns of* $\mathbf{A}$ *are accumulated as specified by the nonzero entries in a column of* $\mathbf{B}$ *using a sparse accumulator or SPA* [24]. *The contents of the SPA are stored into a column of* $\mathbf{C}$ *once all required columns are accumulated.*

Algorithm 1 Columnwise formulation of serial matrix multiplication

```
1: procedure ColumnwiseSpGEMM(A, B, C)
2:     for j ← 1 to n do
3:         for k where B(k, j) ≠ 0 do
4:             C(:, j) ← C(:, j) + A(:, k) · B(k, j)
```

3.1. Distributed memory SpGEMM. The first question for distributed memory algorithms is 'Where is the data?" In parallel SpGEMM, we consider two ways of distributing data to processors. In 1D algorithms, each processor stores a block of m/p rows of an m-by-n sparse matrix. In two-dimensional (2D) algorithms, processors are logically organized as a rectangular $p = p_r \times p_c$ grid, so that a typical processor is named $P(i, j)$. Submatrices are assigned to processors according to a 2D block decomposition: processor $P(i, j)$ stores the submatrix $\mathbf{A}_{ij}$ of dimensions $(m/p_r) \times (n/p_c)$ in its local memory. We extend the colon notation to slices of submatrices: $\mathbf{A}_{i:}$ denotes the $(m/p_r) \times n$ slice of $\mathbf{A}$ collectively owned by all the processors along the ith processor row, and $\mathbf{A}_{:j}$ denotes the $m \times (n/p_c)$ slice of $\mathbf{A}$ collectively owned by all the processors along the jth processor column.

We have previously shown that known 1D SpGEMM algorithms are not scalable to thousands of processors [7], while 2D algorithms can potentially speed up indefinitely, albeit with decreasing efficiency. There are two reasons that the 1D algorithms do not scale: First, their auxiliary data structures cannot be loaded and unloaded quickly enough to amortize their costs. This loading and unloading is necessary because the 1D algorithms proceed in stages in which only one processor broadcasts its submatrix to the others in order to avoid running out of memory. Second, and more fundamentally, the communication costs of 1D algorithms are not scalable regardless of data structures. Each processor receives nnz (of either $\mathbf{A}$ or $\mathbf{B}$) data in the worst case, which implies that communication cost is on the same order as computation, prohibiting speedup beyond a fixed number of processors. This leaves us with 2D algorithms for a scalable solution.

Our previous work [8] shows that the standard compressed sparse column or row (CSC or CSR) data structures are too wasteful for storing the local submatrices arising

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

from a 2D decomposition. This is because the local submatrices are *hypersparse*, meaning that the ratio of nonzeros to dimension is asymptotically zero. The total memory across all processors for CSC format would be $O(n\sqrt{p} + nnz)$, as opposed to $O(n + nnz)$ memory to store the whole matrix in CSC on a single processor. Thus a scalable parallel 2D data structure must respect hypersparsity.

Similarly, any algorithm whose complexity depends on matrix dimension, such as Gustavson's serial SpGEMM algorithm, is asymptotically too wasteful to be used as a computational kernel for multiplying the hypersparse submatrices. Our Hypersparse GEMM [6, 8], on the other hand, operates on the strictly $O(nnz)$ *doubly compressed sparse column* (DCSC) data structure, and its time complexity does not depend on the matrix dimension. Section 3.2 gives a succinct summary of DCSC.

Our HyperSparseGEMM uses an outer-product formulation whose time complexity is $O(nzc(\mathbf{A}) + nzr(\mathbf{B}) + \text{flops} \cdot \lg ni)$, where $nzc(\mathbf{A})$ is the number of columns of $\mathbf{A}$ that are not entirely zero, $nzr(\mathbf{B})$ is the number of rows of $\mathbf{B}$ that are not entirely zero, and ni is the number of indices i for which $\mathbf{A}(:, i) \neq \emptyset$ and $\mathbf{B}(i, :) \neq \emptyset$. The extra $\lg ni$ factor at the time complexity originates from the priority queue that is used to merge ni outer products on the fly. The overall memory requirement of this algorithm is the asymptotically optimal $O(nnz(\mathbf{A}) + nnz(\mathbf{B}) + nnz(\mathbf{C}))$, independent of either matrix dimensions or flops.

3.2. DCSC data structure. DCSC [8] is a further compressed version of CSC where repetitions in the column pointers array, which arise from empty columns, are not allowed. Only columns that have at least one nonzero are represented, together with their column indices.

For example, consider the 9-by-9 matrix with four nonzeros as in Figure 3.3. Figure 3.2 shows its CSC storage, which includes repetitions and redundancies in the column pointers array (CP). Our new data structure compresses this column pointers array to avoid repetitions, giving CP of DCSC as in Figure 3.4. DCSC is essentially a sparse array of sparse columns, whereas CSC is a dense array of sparse columns.

After removing repetitions, CP(i) no longer refers to the ith column. A new JC array, which is parallel to CP, gives us the column numbers. Although our Hypersparse_GEMM algorithm does not need column indexing, DCSC can support fast column indexing by building an AUX array that contains pointers to nonzero columns (columns that have at least one nonzero element) in linear time.

A.I	A.J	A.V
6	1	0.1
8	1	0.2
4	7	0.3
2	8	0.4

Fig. 3.2. *Matrix* $\mathbf{A}$ *in CSC format.*

CP	=	1	3	3	3	3	3	3	4	5	5
		↓						↓	↓		
IR	=	6	8					4	2		
NUM	=	0.1	0.2					0.3	0.4		

Fig. 3.3. *Matrix* $\mathbf{A}$ *in Triples format.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

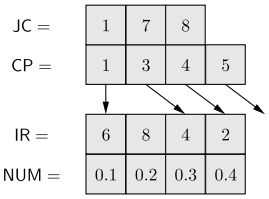


FIG. 3.4. *Matrix* $\mathbf{A}$ *in DCSC format.*

3.3. Sparse SUMMA algorithm. Our parallel algorithm is inspired by the dense matrix-matrix multiplication algorithm SUMMA [23], used in parallel BLAS [16]. SUMMA is memory efficient and easy to generalize to nonsquare matrices and processor grids.

The pseudocode of our 2D algorithm, SPARSESUMMA [7], is shown in Algorithm 2 in its most general form. The coarseness of the algorithm can be adjusted by changing the block size $1 \leq b \leq \gcd(k/p_r, k/p_c)$. For the first time, we present the algorithm in a form general enough to handle rectangular processor grids and a wide range of blocking parameter choices. The pseudocode, however, requires b to evenly divide k/p_r and k/p_c for ease of presentation. This requirement can be dropped at the expense of having potentially multiple broadcasters along a given processor row and column during one iteration of the loop starting at line 4. The **for**... **in parallel do** construct indicates that all of the **do** code blocks execute in parallel by all the processors. The execution of the algorithm on a rectangular grid with rectangular sparse matrices is illustrated in Figure 3.5. We refer the reader to the Combinatorial BLAS source code [2] for additional details.

Algorithm 2 Operation $\mathbf{C} \leftarrow \mathbf{AB}$ using Sparse SUMMA

Input: $\mathbf{A} \in \mathbb{S}^{m \times k}, \mathbf{B} \in \mathbb{S}^{k \times n}$: sparse matrices distributed on a $p_r \times p_c$ processor grid
Output: $\mathbf{C} \in \mathbb{S}^{m \times n}$: the product $\mathbf{AB}$, similarly distributed.

1: **procedure** SPARSESUMMA($\mathbf{A}, \mathbf{B}, \mathbf{C}$)
2: **for** all processors $P(i,j)$ **in parallel do**
3: $\mathbf{B}_{ij} \leftarrow (\mathbf{B}_{ij})^{\mathsf{T}}$
4: **for** $q = 1$ to k/b **do** ▷ blocking parameter b evenly divides k/p_r and k/p_c
5: $c = (q \cdot b)/p_c$ ▷ c is the broadcasting processor column
6: $r = (q \cdot b)/p_r$ ▷ r is the broadcasting processor row
7: $lcols = (q \cdot b) \bmod p_c : ((q+1) \cdot b) \bmod p_c$ ▷ local column range
8: $lrows = (q \cdot b) \bmod p_r : ((q+1) \cdot b) \bmod p_r$ ▷ local row range
9: $\mathbf{A}^{rem} \leftarrow$ BROADCAST($\mathbf{A}_{ic}(:, lcols), P(i,:)$)
10: $\mathbf{B}^{rem} \leftarrow$ BROADCAST($\mathbf{B}_{rj}(lrows, :), P(:,j)$)
11: $\mathbf{C}_{ij} \leftarrow \mathbf{C}_{ij} +$ HYPERSPARSEGEMM($\mathbf{A}^{rem}, \mathbf{B}^{rem}$)
12: $\mathbf{B}_{ij} \leftarrow (\mathbf{B}_{ij})^{\mathsf{T}}$ ▷ Restore the original $\mathbf{B}$

The BROADCAST($\mathbf{A}_{ic}, P(i,:)$) syntax means that the owner of $\mathbf{A}_{ic}$ becomes the root and broadcasts its submatrix to all the processors on the ith processor row. Similarly for BROADCAST($\mathbf{B}_{rj}, P(:,j)$), the owner of $\mathbf{B}_{rj}$ broadcasts its submatrix to all the processors on the jth processor column. In lines 7–8, we find the local column (for $\mathbf{A}$) and row (for $\mathbf{B}$) ranges for matrices that are to be broadcast during that iteration. They are significant only at the broadcasting processors, which can be determined implicitly from the first parameter of BROADCAST. We index $\mathbf{B}$ by

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

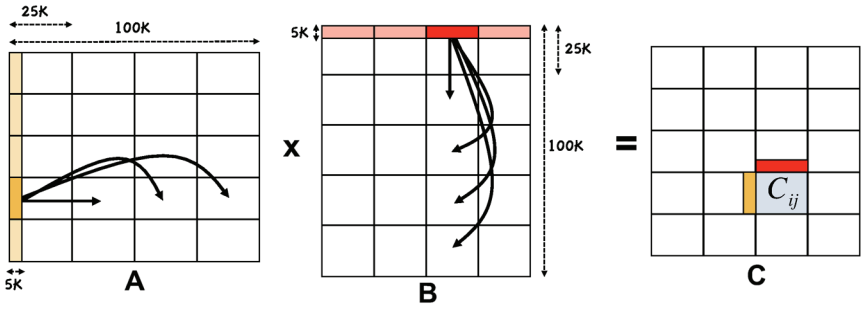


FIG. 3.5. *Execution of the Sparse SUMMA algorithm for sparse matrix-matrix multiplication* $\mathbf{C} = \mathbf{A} \cdot \mathbf{B}$. *The example shows the first stage of the algorithm execution (the broadcast and the local update by processor* $P(i, j)$*). The two rectangular sparse operands* $\mathbf{A}$ *and* $\mathbf{B}$ *are of sizes* m*-by-*$100K$ *and* $100K$*-by-*n*, distributed on a* 5×4 *processor grid. Block size* b *is chosen to be* $5K$.

columns as opposed to rows because it has already been locally transposed in line 3. This makes indexing faster since local submatrices are stored in the column-based DCSC sparse data structure. Using DCSC, the expected cost of fetching b consecutive columns of a matrix $\mathbf{A}$ is b plus the size (number of nonzeros) of the output. Therefore, the algorithm asymptotically has the same computation cost for all values of b.

For our complexity analysis, we assume that nonzeros of input sparse matrices are independently and identically distributed, input matrices are n-by-n, with $d > 0$ nonzeros per row and column on the average. The sparsity parameter d simplifies our analysis by making different terms in the complexity comparable to each other. For example, if $\mathbf{A}$ and $\mathbf{B}$ both have sparsity d, then $nnz(\mathbf{A}) = dn$ and $\text{flops}(\mathbf{AB}) = d^2 n$.

The communication cost of the Sparse SUMMA algorithm, for the case of $p_r = p_c = \sqrt{p}$, is

$$T_{comm} = \sqrt{p}\left(2\,\alpha + \beta\left(\frac{nnz(\mathbf{A}) + nnz(\mathbf{B})}{p}\right)\right) = \Theta\left(\alpha\sqrt{p} + \frac{\beta\, d\, n}{\sqrt{p}}\right), \tag{3.1}$$

and its computation cost is

$$T_{comp} = O\left(\frac{d\,n}{\sqrt{p}} + \frac{d^2 n}{p}\lg\left(\frac{d^2 n}{p\sqrt{p}}\right) + \frac{d^2 n \lg\sqrt{p}}{p}\right) = O\left(\frac{d\,n}{\sqrt{p}} + \frac{d^2 n}{p}\lg\left(\frac{d^2 n}{p}\right)\right) \tag{3.2}$$

(see [6]).

We see that although scalability is not perfect and efficiency deteriorates as p increases, the achievable speedup is not bounded. Since $\lg(d^2 n/p)$ becomes negligible as p increases, the bottlenecks for scalability are the $\beta\, d\, n/\sqrt{p}$ term of T_{comm} and the $d\, n/\sqrt{p}$ term of T_{comp}, which scale with $\sqrt{p}$. Consequently, two different scaling regimes are likely to be present—a close to linear scaling regime, until those terms start to dominate, and a $\sqrt{p}$-scaling regime afterward.

4. Sparse matrix indexing and subgraph selection. Given a sparse matrix $\mathbf{A}$ and two vectors I and J of indices, SpRef extracts a submatrix and stores it as another sparse matrix, $\mathbf{B} = \mathbf{A}(\mathsf{I}, \mathsf{J})$. Matrix $\mathbf{B}$ contains the elements in rows $\mathsf{I}(i)$ and columns $\mathsf{J}(j)$ of $\mathbf{A}$ for $i = 1, \ldots, len(\mathsf{I})$ and $j = 1, \ldots, len(\mathsf{J})$, respecting the order of indices. If $\mathbf{A}$ is the adjacency matrix of a graph, SpRef($\mathbf{A}, \mathsf{I}, \mathsf{I}$) selects an induced subgraph. SpRef can also be used to randomly permute the rows and columns of a

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

sparse matrix, a primitive in parallel matrix computation commonly used for load balancing [31].

Simple cases such as row ($\mathbf{A}(i,:)$), column ($\mathbf{A}(:,i)$), and element ($\mathbf{A}(i,j)$) indexing are often handled by special purpose subroutines [11]. A parallel algorithm for the general case, where I and J are arbitrary vectors of indices, does not exist in the literature. We propose an algorithm that uses parallel SpGEMM. Our algorithm is amenable to performance analysis for the general case.

A related kernel is SpAsgn, or sparse matrix assignment. This operation assigns a sparse matrix to a submatrix of another sparse matrix, $\mathbf{A}(\mathsf{I},\mathsf{J}) = \mathbf{B}$. A variation of SpAsgn is $\mathbf{A}(\mathsf{I},\mathsf{J}) = \mathbf{A}(\mathsf{I},\mathsf{J}) + \mathbf{B}$, which is similar to Liu's *extend-add* operation [30] in finite element matrix assembly. Here we describe the sequential SpAsgn algorithm and its analysis, and we report large-scale performance results in section 5.2.

4.1. Sequential algorithms for SpRef and SpAsgn. Performing SpRef by a triple sparse-matrix product is illustrated in Figure 4.1. The algorithm can be described concisely in MATLAB notation as follows:

```
function B = spref(A,I,J)

[m,n] = size(A);
R = sparse(1:len(I),I,1,len(I),m);
Q = sparse(J,1:len(J),1,n,len(J));
B = R*A*Q;
```

The sequential complexity of this algorithm is $\text{flops}(\mathbf{R}\cdot\mathbf{A}) + \text{flops}((\mathbf{RA})\cdot\mathbf{Q})$. Due to the special structure of the permutation matrices, the number of nonzero operations required to form the product $\mathbf{R}\cdot\mathbf{A}$ is equal to the number of nonzero elements in the product. That is, $\text{flops}(\mathbf{R}\cdot\mathbf{A}) = nnz(\mathbf{RA}) \le nnz(\mathbf{A})$. Similarly, $\text{flops}((\mathbf{RA})\cdot\mathbf{Q}) \le nnz(\mathbf{A})$, making the overall complexity $O(nnz(\mathbf{A}))$ for any I and J. This is optimal in general, since just writing down the result of a matrix permutation $\mathbf{B} = \mathbf{A}(r,r)$ requires $\Omega(nnz(\mathbf{A}))$ operations.

Performing SpAsgn by two triple sparse-matrix products and additions is illustrated in Figure 4.2. We create two temporary sparse matrices of the same dimensions as $\mathbf{A}$. These matrices contain nonzeros only for the $\mathbf{A}(\mathsf{I},\mathsf{J})$ part and zeros elsewhere. The first triple product embeds $\mathbf{B}$ into a bigger sparse matrix that we add to $\mathbf{A}$. The second triple product embeds $\mathbf{A}(\mathsf{I},\mathsf{J})$ into an identically sized sparse matrix so that we can zero out the $\mathbf{A}(\mathsf{I},\mathsf{J})$ portion by subtracting it from $\mathbf{A}$. Since general semiring axioms do not require additive inverses to exist, we implement this piece of the algorithm slightly differently than stated in the pseudocode. We still form the **SAT**

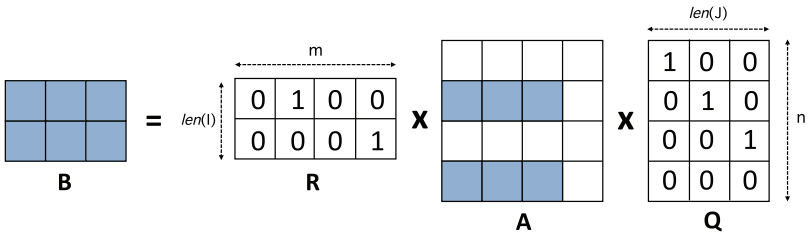


FIG. 4.1. *Sparse matrix indexing (SpRef) using mixed-mode SpGEMM. On an m-by-n matrix* $\mathbf{A}$*, the SpRef operation* $\mathbf{A}(\mathsf{I},\mathsf{J})$ *extracts a* len(I)*-by-*len(J) *submatrix, where* I *is a vector of row indices and* J *is a vector of column indices. The example shows* $\mathbf{B} = \mathbf{A}([2,4],[1,2,3])$*. It performs two SpGEMM operations between a boolean matrix and a general-type matrix.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

$$\mathbf{A} = \mathbf{A} + \begin{pmatrix} 0 & 0 & 0 \\ 0 & \mathbf{B} & 0 \\ 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 \\ 0 & \mathbf{A}(\mathsf{I},\mathsf{J}) & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

FIG. 4.2. *Illustration of SpAsgn* ($\mathbf{A}(\mathsf{I},\mathsf{J}) = \mathbf{B}$) *for rings where additive inverses are defined. For simplicity, the vector indices* I *and* J *are shown as contiguous, but they need not be.*

product, but, instead of using subtraction, we use the generalized sparse elementwise multiplication function of the Combinatorial BLAS [10] to zero out the $\mathbf{A}(\mathsf{I},\mathsf{J})$ portion. In particular, we first perform an elementwise multiplication of $\mathbf{A}$ with the negation of $\mathbf{SAT}$ without explicitly forming the negated matrix, which can be dense. Thanks to this direct support for the implicit negation operation, the complexity bounds are identical to the version that uses subtraction. The negation does not assume additive inverses: it sets all zero entries to one and all nonzeros entries to zero. The algorithm can be described concisely in MATLAB notation as follows:

```
function C = spasgn(A,I,J,B)
% A = spasgn(A,I,J,B) performs A(I,J) = B

[ma,na] = size(A);
[mb,nb] = size(B);
R = sparse(I,1:mb,1,ma,mb);
Q = sparse(1:nb,J,1,nb,na);
S = sparse(I,I,1,ma,ma);
T = sparse(J,J,1,na,na);
C = A + R*B*Q - S*A*T;
```

Liu's *extend-add* operation is similar to SpAsgn but simpler; it just omits subtracting the $\mathbf{SAT}$ term.

Let us analyze the complexity of SpAsgn. Given $\mathbf{A} \in \mathbb{S}^{m \times n}$ and $\mathbf{B} \in \mathbb{S}^{len(\mathsf{I}) \times len(\mathsf{J})}$, the intermediate boolean matrices have the following properties:

$\mathbf{R}$ is m-by-$len(\mathsf{I})$ rectangular with $len(\mathsf{I})$ nonzeros, one in each column.

$\mathbf{Q}$ is $len(\mathsf{J})$-by-n rectangular with $len(\mathsf{J})$ nonzeros, one in each row.

$\mathbf{S}$ is m-by-m symmetric with $len(\mathsf{I})$ nonzeros, all located along the diagonal.

$\mathbf{T}$ is n-by-n symmetric with $len(\mathsf{J})$ nonzeros, all located along the diagonal.

THEOREM 4.1. *The sequential SpAsgn algorithm takes* $O(nnz(\mathbf{A}) + nnz(\mathbf{B}) + len(\mathsf{I}) + len(\mathsf{J}))$ *time using an optimal* $\Theta(\text{flops})$ *SpGEMM subroutine.*

Proof. The product $\mathbf{R} \cdot \mathbf{B}$ requires $\text{flops}(\mathbf{R} \cdot \mathbf{B}) = nnz(\mathbf{RB}) = nnz(\mathbf{B})$ operations because there is a one-to-one relationship between nonzeros in the output and flops performed. Similarly, $\text{flops}((\mathbf{RB}) \cdot \mathbf{Q}) = nnz(\mathbf{RBQ}) = nnz(\mathbf{B})$, yielding $\Theta(nnz(\mathbf{B}))$ complexity for the first triple product. The product $\mathbf{S} \cdot \mathbf{A}$ requires only $len(\mathsf{I})$ flops since it does not need to touch nonzeros of $\mathbf{A}$ that do not contribute to $\mathbf{A}(\mathsf{I},:)$. Similarly, $(\mathbf{SA}) \cdot \mathbf{T}$ requires only $len(\mathsf{J})$ flops. The number of nonzeros in the second triple product is $nnz(\mathbf{SAT}) = O(len(\mathsf{I}) + len(\mathsf{J}))$. The final pointwise addition and subtraction (or generalized elementwise multiplication in the absence of additive inverses) operations take time on the order of the total number of nonzeros in all operands [11], which is $O(nnz(\mathbf{A}) + nnz(\mathbf{B}) + len(\mathsf{I}) + len(\mathsf{J}))$. ∎

4.2. SpRef in parallel. The parallelization of SpRef poses several challenges. The boolean matrices have only one nonzero per row or column. For the parallel 2D algorithm to scale well with an increasing number of processors, data structures and algorithms should respect hypersparsity [8]. Communication should ideally take place along a single processor dimension to save a factor of $\sqrt{p}$ in communication

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

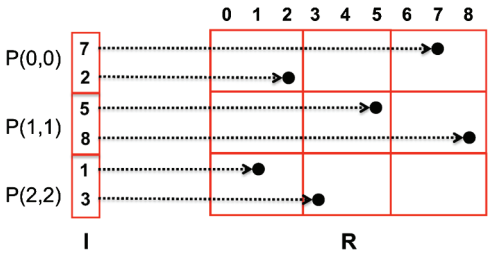


FIG. 4.3. *Parallel forming of the left-hand side boolean matrix* $\mathbf{R}$ *from the index vector* I *on nine processors in a logical* 3×3 *grid.* $\mathbf{R}$ *will be subsequently multiplied with* $\mathbf{A}$ *to extract six rows out of nine from* $\mathbf{A}$ *and order them as* $\{7, 2, 5, 8, 1, 3\}$.

volume. As before, we assume a uniform distribution of nonzeros to processors in our analysis.

The communication cost of forming the $\mathbf{R}$ matrix in parallel is the cost of SCATTER along the processor column. For the case of vector I distributed to $\sqrt{p}$ diagonal processors, scattering can be implemented with an average communication cost of $\Theta(\alpha \cdot \lg p + \beta \cdot (len(\mathsf{I})/\sqrt{p})$ [14]. This process is illustrated in Figure 4.3. The $\mathbf{Q}^\mathsf{T}$ matrix can be constructed identically, followed by a TRANSPOSE($\mathbf{Q}^\mathsf{T}$) operation where each processor $P(i, j)$ receives $nnz(\mathbf{Q})/p = len(\mathsf{J})/p$ words of data from its diagonal neighbor $P(j, i)$. Note that the communication cost of the transposition is dominated by the cost of forming $\mathbf{Q}^\mathsf{T}$ via SCATTER.

While the analysis of our parallel SpRef algorithm assumes that the index vectors are distributed only on diagonal processors, the asymptotic costs are identical in the 2D case, where the vectors are distributed across all the processors [12]. This is because the number of elements (the amount of data) received by a given processor stays the same with the only difference in the algorithm being the use of the ALLTOALL operation instead of SCATTER during the formation of the $\mathbf{R}$ and $\mathbf{Q}$ matrices.

The parallel performance of SpGEMM is a complicated function of the matrix nonzero structures [7, 9]. For SpRef, however, the special structure makes our analysis more precise. Suppose that the triple product is evaluated from left to right: $\mathbf{B} = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{Q}$. A similar analysis can be applied to the reverse evaluation. A conservative estimate of $ni(\mathbf{R}, \mathbf{A})$, the number of indices i for which $\mathbf{R}(:, i) \neq \emptyset$ and $\mathbf{A}(i, :) \neq \emptyset$, is $nnz(\mathbf{R}) = len(\mathsf{I})$.

Using our HYPERSPARSEGEMM [6, 8] as the computational kernel, time to compute the product $\mathbf{RA}$ (excluding the communication costs) is

$$T_{mult} = \max_{i,j} \sum_{k=1}^{\sqrt{p}} \big(nzc(\mathbf{R}_{ik}) + nzr(\mathbf{A}_{kj}) + \text{flops}(\mathbf{R}_{ik} \cdot \mathbf{A}_{kj}) \cdot \lg ni(\mathbf{R}_{ik}, \mathbf{A}_{kj}) \big),$$

where the maximum over all (i, j) pairs is equal to the average, due to the uniform nonzero distribution assumption.

Recall from the sequential analysis that $\text{flops}(\mathbf{R} \cdot \mathbf{A}) \leq nnz(\mathbf{A})$ since each nonzero in $\mathbf{A}$ contributes at most once to the overall flop count. We also know that $nzc(\mathbf{R}) = len(\mathsf{I})$ and $nzr(\mathbf{A}) \leq nnz(\mathbf{A})$. Together with the uniformity assumption, these identities yield the following results:

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

$$\text{flops}(\mathbf{R}_{ik} \cdot \mathbf{A}_{kj}) = \frac{nnz(\mathbf{A})}{p\sqrt{p}},$$
$$ni(\mathbf{R}_{ik} \cdot \mathbf{A}_{kj}) \leq nnz(\mathbf{R}_{ik}) = \frac{len(\mathsf{I})}{p},$$
$$\sum_{k=1}^{\sqrt{p}} nzc(\mathbf{R}_{ik}) = nzc(\mathbf{R}_{i:}) = \frac{len(\mathsf{I})}{\sqrt{p}},$$
$$\sum_{k=1}^{\sqrt{p}} nzr(\mathbf{A}_{ik}) = nzr(\mathbf{A}_{i:}) \leq \frac{nnz(\mathbf{A})}{\sqrt{p}}.$$

In addition to the multiplication costs, adding intermediate triples in $\sqrt{p}$ stages costs an extra $\text{flops}(\mathbf{R}_{i:} \cdot \mathbf{A}_{:j}) \lg \sqrt{p} = (nnz(\mathbf{A})/p) \lg \sqrt{p}$ operations per processor. Thus, we have the following estimates of computation and communication costs for computing the product $\mathbf{RA}$:

$$T_{comp}(\mathbf{R} \cdot \mathbf{A}) = O\left(\frac{len(\mathsf{I}) + nnz(\mathbf{A})}{\sqrt{p}} + \frac{nnz(\mathbf{A})}{p} \cdot \lg\left(\frac{len(\mathsf{I})}{p} + \sqrt{p}\right)\right),$$
$$T_{comm}(\mathbf{R} \cdot \mathbf{A}) = \Theta\left(\alpha \cdot \sqrt{p} + \beta \cdot \frac{nnz(\mathbf{A})}{\sqrt{p}}\right).$$

Given that $nnz(\mathbf{RA}) \leq nnz(\mathbf{A})$, the analysis of multiplying the intermediate product $\mathbf{RA}$ with $\mathbf{Q}$ is similar. Combined with the cost of forming auxiliary matrices $\mathbf{R}$ and $\mathbf{Q}$ and the costs of transposition of $\mathbf{Q}^{\mathsf{T}}$, the total cost of the parallel SpRef algorithm becomes

$$T_{comp} = O\left(\frac{len(\mathsf{I}) + len(\mathsf{J}) + nnz(\mathbf{A})}{\sqrt{p}} + \frac{nnz(\mathbf{A})}{p} \cdot \lg\left(\frac{len(\mathsf{I}) + len(\mathsf{J})}{p} + \sqrt{p}\right)\right),$$
$$T_{comm} = \Theta\left(\alpha \cdot \sqrt{p} + \beta \cdot \frac{nnz(\mathbf{A}) + len(\mathsf{I}) + len(\mathsf{J})}{\sqrt{p}}\right).$$

We see that SpGEMM costs dominate the cost of SpRef. The asymptotic speedup is limited to $\Theta(\sqrt{p})$, as in the case of SpGEMM.

5. Experimental results. We ran experiments on NERSC's Franklin system [1], a 9660-node Cray XT4. Each XT4 node contains a quad-core 2.3 GHz AMD Opteron processor, attached to the XT4 interconnect via a Cray SeaStar2 ASIC using a HyperTransport 2 interface capable of 6.4 GB/s. The SeaStar2 routing ASICs are connected in a three-dimensional torus topology, and each link is capable of 7.6 GB/s peak bidirectional bandwidth. Our algorithms perform similarly well on a fat tree topology, as evidenced by our experimental results on the Ranger platform that are included in an earlier technical report [9].

We used the GNU C/C++ compilers (version 4.5) and Cray's MPI implementation, which is based on MPICH2. We incorporated our code into the Combinatorial BLAS framework [10]. We experimented with core counts that are perfect squares, because the Combinatorial BLAS currently uses a square $\sqrt{p} \times \sqrt{p}$ processor grid. We compared performance with the Trilinos package (version 10.6.2.0) [28], which uses a 1D decomposition for its sparse matrices.

In the majority of our experiments, we used synthetically generated R-MAT matrices rather than Erdős–Rényi [22] "flat" random matrices, as these are more realistic

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

for many graph analysis applications. R-MAT [13], the Recursive MATrix generator, generates graphs with skewed degree distributions that approximate a power-law. A scale n R-MAT matrix is 2^n-by-2^n. Our R-MAT matrices have an average of eight nonzeros per row and column. R-MAT seed parameters are $a = .6$, and $b = c = d = .4/3$. We applied a random symmetric permutation to the input matrices to balance the memory and the computational load. In other words, instead of storing and computing $\mathbf{C} = \mathbf{AB}$, we compute $\mathbf{PCP}^\mathsf{T} = (\mathbf{PAP}^\mathsf{T})(\mathbf{PBP}^\mathsf{T})$. All of our experiments are performed on double-precision floating-point inputs.

Since algebraic multigrid on graphs coming from physical problems is an important case, we included two more matrices from the Florida Sparse Matrix collection [21] to our experimental analysis in section 5.3.2, where we benchmark a restriction operation that is used in algebraic multigrid. The first such matrix is a large circuit problem (Freescale1) with 17 million nonzeros and 3.42 million rows and columns. The second matrix comes from a structural problem (GHS_psdef/ldoor) and has 42.5 million nonzeros and 952,203 rows and columns.

5.1. Parallel scaling of SpRef. Our first set of experiments randomly permutes the rows and columns of $\mathbf{A}$ as an example case study for matrix reordering and partitioning. This operation corresponds to relabeling vertices of a graph. Our second set of experiments explores subgraph extraction by generating a random permutation of $1 : n$ and dividing it into $k \ll n$ chunks $r_1, \ldots, r_k$. We then performed k SpRef operations of the form $\mathbf{A}(r_i, r_i)$, one after another (with a barrier in between). In both cases, the sequential reference is our algorithm itself.

The performance and parallel scaling of the symmetric random permutation is shown in Figure 5.1. The input is an R-MAT matrix of scale 22 with approximately 32 million nonzeros in a square matrix of dimension 2^{22}. Speedup and runtime are plotted on different vertical axes. We see that scaling is close to linear up to about 64 processors, and proportional to $\sqrt{p}$ afterward, agreeing with our analysis.

The performance of subgraph extraction for $k = 10$ induced subgraphs, each with n/k randomly chosen vertices, is shown in Figure 5.2. The algorithm performs well in this case, too. The observed scaling is slightly less than the case of applying a single big permutation, which is to be expected since the multiple small subgraph extractions increase span and decrease available parallelism.

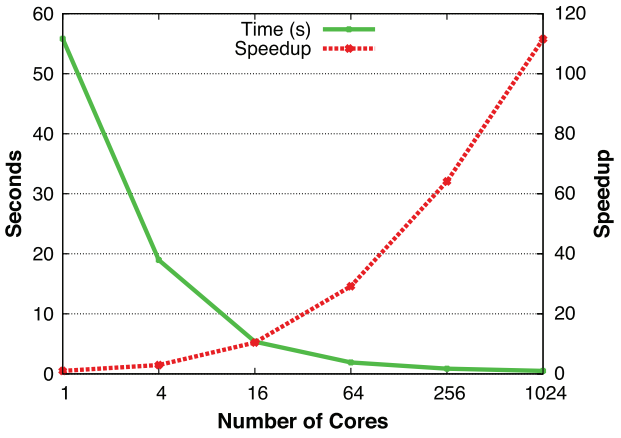


FIG. 5.1. *Performance and parallel scaling of applying a random symmetric permutation to an R-MAT matrix of scale* 22. *The x-axis uses a log scale.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

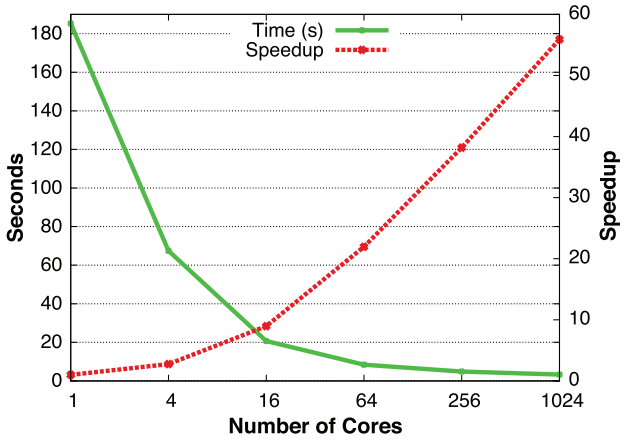


FIG. 5.2. *Performance and parallel scaling of extracting* 10 *induced subgraphs from an R-MAT matrix of scale* 22. *The x-axis uses a log scale.*

5.2. Parallel scaling of SpAsgn. We benchmarked our parallel SpAsgn code by replacing a portion of the input matrix (**A**) with a structurally similar right-hand side matrix (**B**). This operation is akin to replacing a portion of the graph due to a streaming update. The subset of vertices (row and column indices of **A**) to be updated is chosen randomly. In all the tests, the original graph is an R-MAT matrix of scale 22 with 32 million nonzeros. The right-hand side (replacement) matrix is also an R-MAT matrix of scales 21, 20, and 19, in three subsequent experiments, replacing 50%, 25%, and 12.5% of the original graph, respectively. The average numbers of nonzeros per row and column are also adjusted for the right-hand side matrices to match the nonzero density of the subgraphs they are replacing.

The performance of this sparse matrix assignment operation is shown in Figure 5.3. Our implementation uses a small number of Combinatorial BLAS routines: these are a sparse matrix constructor from distributed vectors, essentially a parallel version of MATLAB's `sparse` routine, the generalized elementwise multiplication with direct support for negation, and parallel SpGEMM implemented using Sparse SUMMA.

5.3. Parallel scaling of sparse SUMMA. We implemented two versions of the 2D parallel SpGEMM algorithms in C++ using MPI. The first is directly based on Sparse SUMMA and is synchronous in nature, using blocking broadcasts. The second is asynchronous and uses one-sided communication in MPI-2. We found the asynchronous implementation to be consistently slower than the broadcast-based synchronous implementation due to inefficient implementation of one-sided communication routines in MPI. Therefore, we report only the performance of the synchronous implementation. The motivation behind the asynchronous approach, performance comparisons, and implementation details can be found in our technical report [9, section 7]. On more than four cores of Franklin, synchronous implementation consistently outperformed the asynchronous implementation by 38–57%.

Our sequential HYPERSPARSEGEMM routines return a set of intermediate triples that are kept in memory up to a certain threshold without being merged immediately. This permits more balanced merging, eliminating some unnecessary scans that degraded performance in a preliminary implementation [7].

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

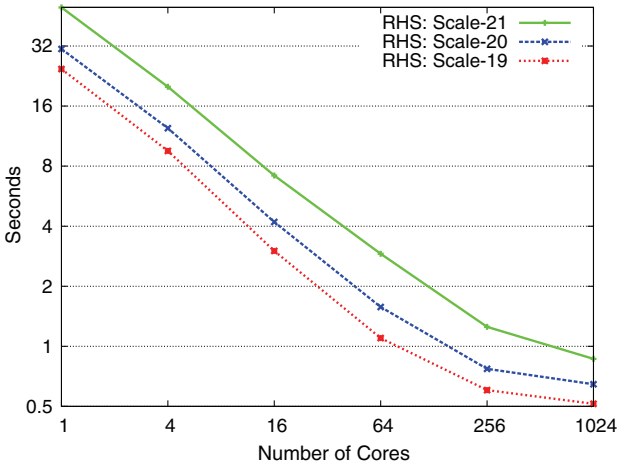


FIG. 5.3. *Observed scaling of the SpAsgn operation* $\mathbf{A}(\mathsf{I},\mathsf{I}) = \mathbf{B}$ *where* $\mathbf{A} \in \mathbb{R}^{n \times n}$ *is an R-MAT matrix of scale* 22 *and* $\mathbf{B}$ *is another R-MAT matrix whose scale is shown in the figure legend.* I *is a duplicate-free sequence with entries randomly selected from the range* 1...n*; its length matches the dimensions of* $\mathbf{B}$*. Both axes are log scale.*

5.3.1. Square sparse matrix multiplication. In the first set of experiments, we multiply two structurally similar R-MAT matrices. This square multiplication is representative of the expansion operation used in the Markov clustering algorithm [36]. It is also a challenging case for our implementation due to the highly skewed nonzero distribution. We performed strong scaling experiments for matrix dimensions ranging from 2^{21} to 2^{24}.

Figure 5.4 shows the speedup we achieved. The graph shows linear speedup until around 100 processors; afterward the speedup is proportional to the square root of the number of processors. Both results agree with the theoretical analysis. To illustrate how the scaling transitions from linear to $\sqrt{p}$, we drew trend lines on the scale 21 results. As shown in Figure 5.5, the slope of the log-log curve is 0.85 (close to linear)

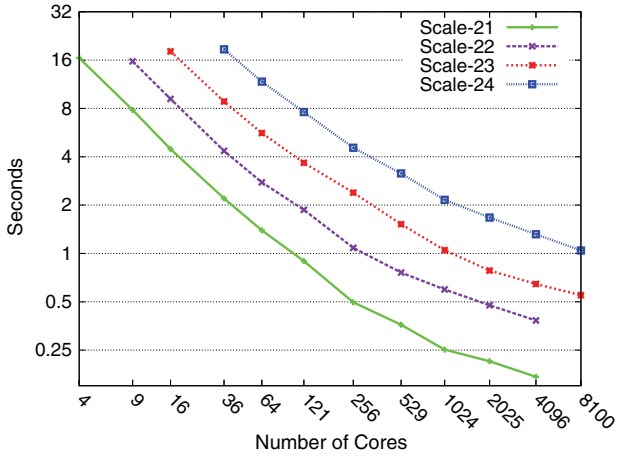


FIG. 5.4. *Observed scaling of synchronous Sparse SUMMA for the* R-MAT × R-MAT *product on matrices having dimensions* $2^{21} - 2^{24}$*. Both axes are log scale.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

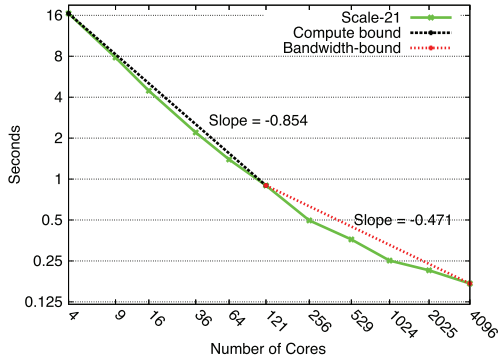


FIG. 5.5. *Demonstration of two scaling regimes for scale* 21 *R-MAT product.*

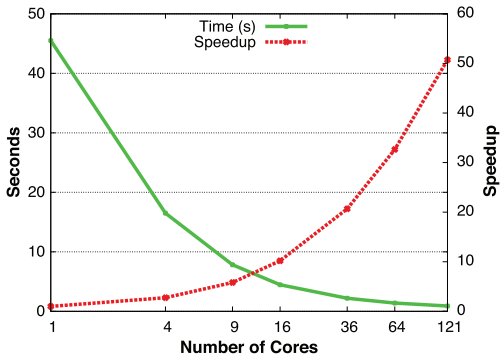


FIG. 5.6. *Performance and scaling of Sparse SUMMA at lower concurrencies (scale* 21 *inputs). The x-axis uses a log scale.*

until 121 cores, and the slope afterward is 0.47 (close to $\sqrt{p}$). Figure 5.6 zooms to the linear speedup regime and shows the performance of our algorithm at lower concurrencies. The speedup and timings are plotted on different y-axes of the same graph.

Our implementation of Sparse SUMMA achieves over 2 billion "useful flops" (in double precision) per second on 8100 cores when multiplying scale 24 R-MAT matrices. Since useful flops are highly dependent on the matrix structure and sparsity, we provide additional statistics for this operation in Table 5.1. Using matrices with more nonzeros per row and column will certainly yield higher performance rates (in useful flops). The gains from sparsity are clear if one considers dense flops that would be needed if these matrices were stored in a dense format. For example, multiplying two dense scale 24 matrices requires 9444 exaflops.

Figure 5.7 breaks down the time spent in communication and computation when multiplying two R-MAT graphs of scale 24. We see that computation scales much better than communication (over 90x reduction when going from 36 to 8100 cores), implying that SpGEMM is communication bound for large concurrencies. For example, on 8100 cores, 83% of the time is spent in communication. Communication times include the overheads due to synchronization and load imbalance.

Figure 5.7 also shows the effect of different blocking sizes. Remember that each processor owns a submatrix of size $n/\sqrt{p}$-by-$n/\sqrt{p}$. On the left, the algorithm com-

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

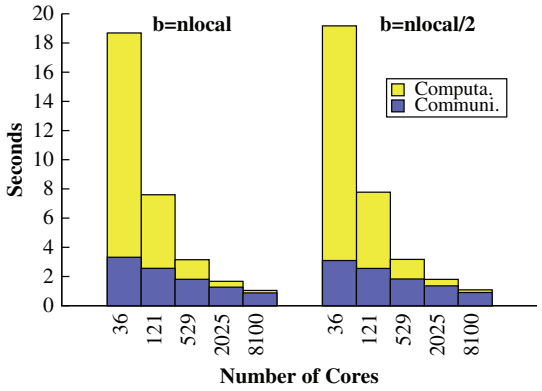


FIG. 5.7. *Communication and computation breakdown, at various concurrencies and two blocking sizes (scale* 24 *inputs).*

TABLE 5.1
Statistics about R-MAT product $\mathbf{C} = \mathbf{A} \cdot \mathbf{B}$. *All numbers (except scale) are in millions.*

Scale	$nnz(\mathbf{A})$	$nnz(\mathbf{B})$	$nnz(\mathbf{C})$	flops
21	16.3	16.3	123.9	253.2
22	32.8	32.8	257.1	523.7
23	65.8	65.8	504.3	1021.3
24	132.1	132.1	1056.9	2137.4

pletes in $\sqrt{p}$ stages, each time broadcasting its whole local matrix. On the right, the algorithm completes in $2\sqrt{p}$ stages, each time broadcasting half of its local matrix. We see that while communication costs are not affected, the computation slows down by 1–6% when doubling the number of stages. This difference is due to the costs of splitting the input matrices before the multiplication and reassembling them afterward, which is small because splitting and reassembling are simple scans over the data whose costs are dominated by the cost of multiplication itself.

5.3.2. Multiplication with the restriction operator. Multilevel methods are widely used in the solution of numerical and combinatorial problems [35]. Such methods construct smaller problems by successive coarsening. The simplest coarsening is graph contraction: a contraction step chooses two or more vertices in the original graph G to become a single aggregate vertex in the contracted graph G'. The edges of G that used to be incident to any of the vertices forming the aggregate become incident to the new aggregate vertex in G'.

Constructing a coarse grid during the V-cycle of algebraic multigrid [5] or graph partitioning [27] is a generalized graph contraction operation. Different algorithms need different coarsening operators. For example, a weighted aggregation [15] might be preferred for partitioning problems. In general, coarsening can be represented as multiplication of the matrix representing the original fine domain (grid, graph, or hypergraph) by the restriction operator.

In these experiments, we use a simple restriction operation to perform graph contraction. Gilbert, Reinhardt, and Shah [25] describe how to perform contraction using SpGEMM. Their algorithm creates a special sparse matrix $\mathbf{S}$ with n nonzeros. The triple product $\mathbf{SAS}^{\mathsf{T}}$ contracts the whole graph at once. Making $\mathbf{S}$ smaller in the

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

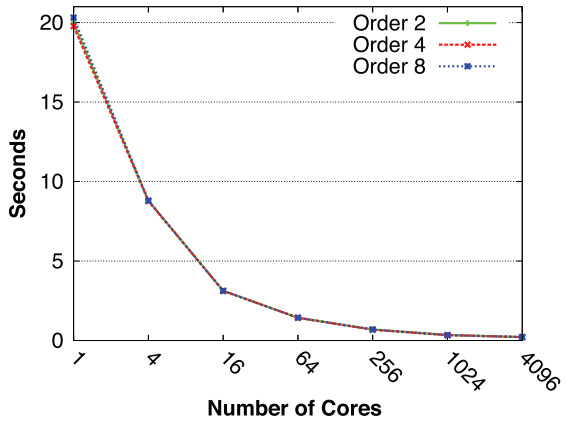


FIG. 5.8. *Strong scaling of* $\mathbf{B} \leftarrow \mathbf{AS}^{\mathsf{T}}$, *multiplying scale* 23 *R-MAT matrices with the restriction operator on the right. The x-axis uses a log scale.*

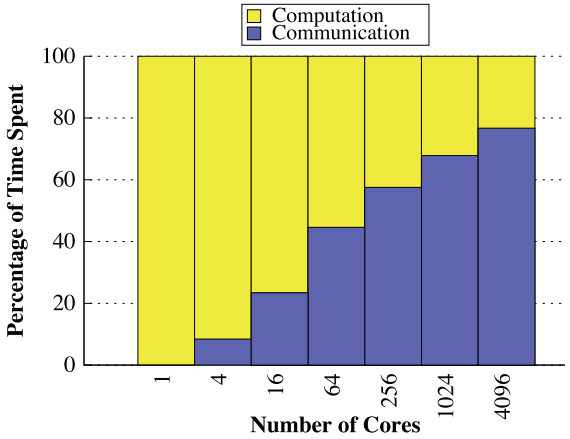


FIG. 5.9. *Normalized communication and computation breakdown for multiplying scale* 23 *R-MAT matrices with the restriction operator of order* 4.

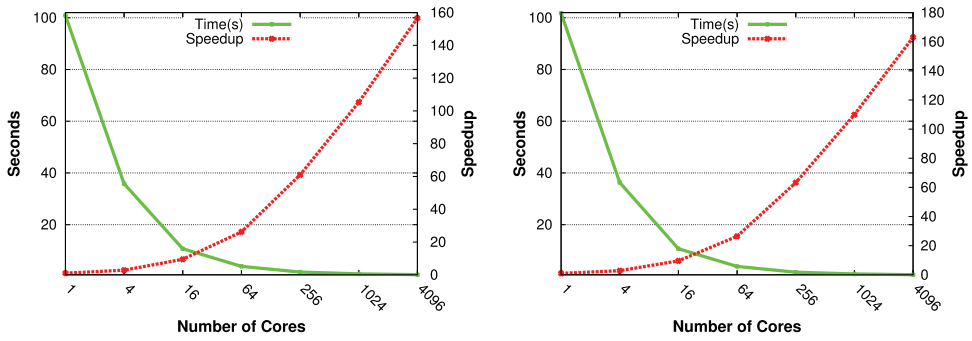
first dimension while keeping the number of nonzeros the same changes the restriction order. For example, we contract the graph by half by using $\mathbf{S}$ having dimensions $n/2 \times n$, which is said to be of order 2.

Figure 5.8 shows "strong scaling" of the $\mathbf{AS}^{\mathsf{T}}$ operation for R-MAT graphs of scale 23. We used restrictions of order 2, 4, and 8. Changing the interpolation order results in minor (less than 5%) changes in performance, as shown by the overlapping curves. This is further evidence that our algorithm's complexity is independent of the matrix dimension, because interpolation order has a profound effect on the dimension of the right-hand side matrix, but it does not change the expected flops and numbers of nonzeros in the inputs (it may slightly decrease the number of nonzeros in the output). The experiment shows scaling up to 4096 processors. Figure 5.9 shows the breakdown of time (as percentages) spent on remote communication and local computation steps.

Figures 5.10a and 5.10b show strong scaling of the full restriction operation $\mathbf{SAS}^{\mathsf{T}}$ of order 8, using different parenthesizing for the triple product. The results show that

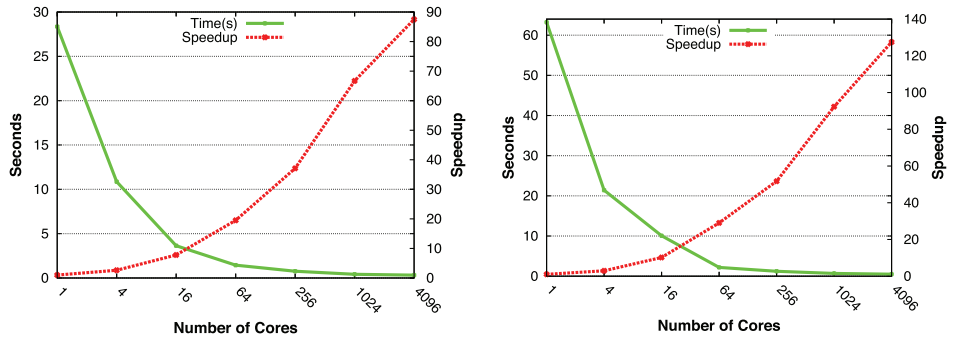
Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy



(a) Left to right evaluation: $(\mathbf{SA})\mathbf{S}^\mathsf{T}$ (b) Right to left evaluation: $\mathbf{S}(\mathbf{AS}^\mathsf{T})$

FIG. 5.10. *The full restriction operation of order* 8 *applied to a scale* 23 *R-MAT matrix.*



(a) Freescale1 matrix. (b) GHS_psdef/ldoor matrix.

FIG. 5.11. *Performance and strong scaling of Sparse SUMMA implementation for the full restriction of order* 2 ($\mathbf{SAS}^\mathsf{T}$) *on real matrices from physical problems.*

our code achieves 110× speedup on 1024-way concurrency and 163× speedup on 4096-way concurrency, and the performance is not affected by the different parenthesizing.

Figure 5.11 shows the performance of full operation on real matrices from physical problems. Both matrices have a full diagonal that remains full after symmetric permutation. Due to the 2D decomposition, processors responsible for the diagonal blocks typically have more work to do. For load-balancing and performance reasons, we split these matrices into two pieces: $\mathbf{A} = \mathbf{D} + \mathbf{L}$, where $\mathbf{D}$ is the diagonal piece and $\mathbf{L}$ is the off-diagonal piece. The restriction of rows becomes $\mathbf{SA} = \mathbf{SD} + \mathbf{SL}$. Scaling the columns of $\mathbf{S}$ with the diagonal of $\mathbf{D}$ performs the former multiplication, and the latter multiplication uses the Sparse SUMMA algorithm described in our paper. This splitting approach especially improved the scalability of restriction on the Freescale1 matrix, because it is much sparser than GHS_psdef/ldoor, which does not face severe load balancing issues. Order 2 restriction shrinks the number of nonzeros from 17.0 to 15.3 million for Freescale1, and from 42.5 to 42.0 million for GHS_psdef/ldoor.

5.3.3. Tall skinny right-hand side matrix. The last set of experiments multiplies R-MAT matrices by tall skinny matrices of varying sparsity. This computation is representative of the parallel breadth-first search that lies at the heart of our distributed-memory betweenness centrality implementation [10]. This set indirectly

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

examines the sensitivity to sparsity as well, because we vary the sparsity of the right-hand side matrix from approximately 1 to 10^5 nonzeros per column in powers of 10. In this way, we imitate the patterns of the level-synchronous breadth-first search from multiple source vertices where the current frontier can range from a few vertices to hundreds of thousands [12].

For our experiments, the R-MAT matrices on the left-hand side have $d_1 = 8$ nonzeros per column, and their dimensions vary from $n = 2^{20}$ to $n = 2^{26}$. The right-hand side is an Erdős–Rényi matrix of dimensions n-by-k, and the number of nonzeros per column, d_2, is varied from 1 to 10^5, in powers of 10. The right-hand side matrix's width k varies from 128 to 8192, growing proportionally to its length n, hence keeping the matrix aspect ratio constant at $n/k = 8192$. Except for the $d_2 = 10^5$ case, the R-MAT matrix has more nonzeros than the right-hand side matrix. In this computation, the total work is $W = O(d_1 d_2 k)$, the total memory consumption is $M = O(d_1 n + d_2 k)$, and the total bandwidth requirement is $O(M\sqrt{p})$.

We performed weak scaling experiments where memory consumption per processor is constant. Since $M = O(d_1 n + d_2 k)$, this is achieved by keeping both $n/p = 2^{14}$ and $k/p = 2$ constant. Work per processor is also constant. However, per-processor bandwidth requirements of this algorithm increases by a factor of $\sqrt{p}$.

Figure 5.12 shows a performance graph in three dimensions. The timings for each slice along the XZ-plane (i.e., for every $d_2 = \{1, 10, \ldots, 10^5\}$ contour) are normalized to the running time on 64 processors. We do not cross-compare the absolute performances for different d_2 values, as our focus in this section is parallel scaling. In line with the theory, we observe the expected $\sqrt{p}$ slowdown due to communication costs.

The performance we achieved for these large-scale experiments, where we ran our code on up to 4096 processors, is remarkable. It also shows that our implementation does not incur any significant overhead since it does not deviate from the $\sqrt{p}$ curve.

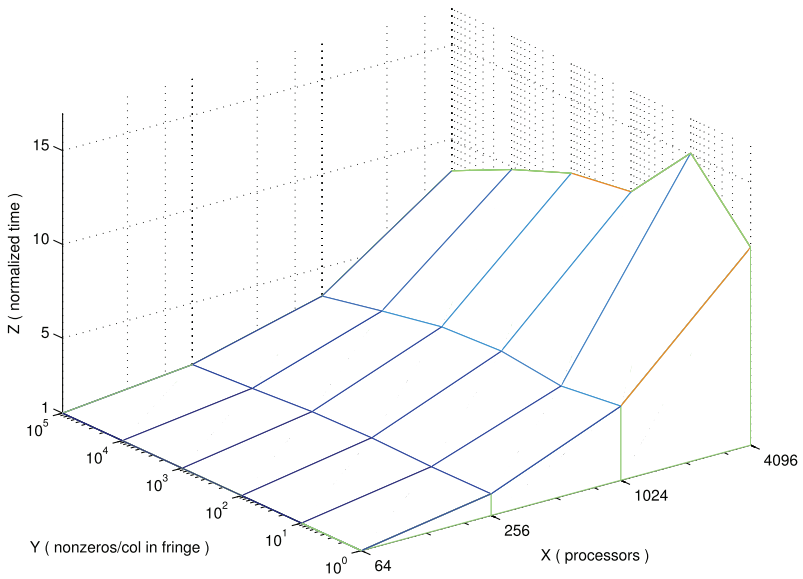
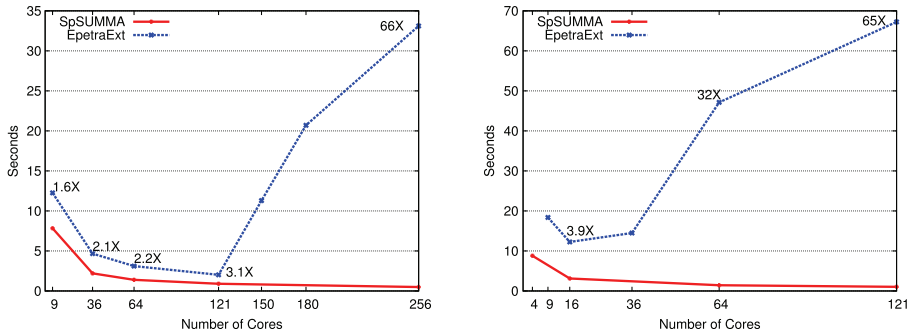


FIG. 5.12. *Weak scaling of R-MAT times a tall skinny Erdős–Rényi matrix.* X *(processors) and* Y *(nonzeros per column on fringe) axes are logarithmic, whereas* Z *(normalized time) axis is linear.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

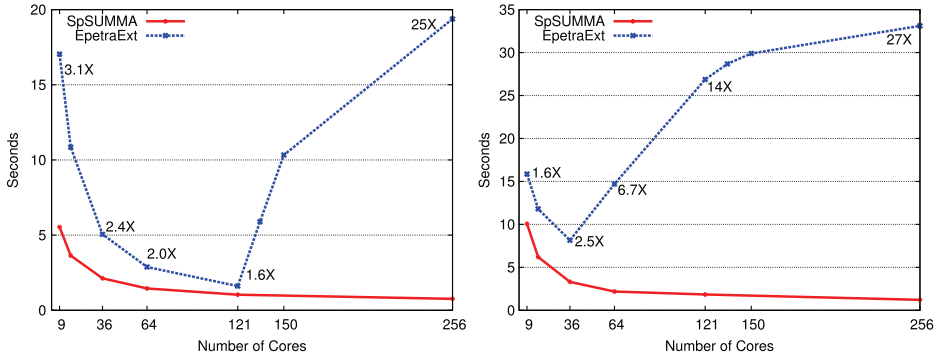
Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy



(a) R-MAT × R-MAT product (scale 21).

(b) Multiplication of an R-MAT matrix of scale 23 with the restriction operator of order 8.

FIG. 5.13. *Comparison of SpGEMM implementation of Trilinos's EpetraExt package with our Sparse SUMMA implementation using synthetically generated matrices. The data labels on the plots show the speedup of Sparse SUMMA over EpetraExt.*



(a) Freescale matrix.

(b) GHS_psdef/ldoor matrix.

FIG. 5.14. *Comparison of Trilinos's EpetraExt package with our Sparse SUMMA implementation for the full restriction of order* 2 (SAS^T) *on real matrices. The data labels on the plots show the speedup of Sparse SUMMA over EpetraExt.*

5.4. Comparison with Trilinos. The EpetraExt package of Trilinos can multiply two distributed sparse matrices in parallel. Trilinos can also permute matrices and extract submatrices through its Epetra_Import and Epetra_Export classes. These packages of Trilinos use a 1D data layout.

For SpGEMM, we compared the performance of Trilinos's EpetraExt package with ours on two scenarios. In the first scenario, we multiplied two R-MAT matrices as described in section 5.3.1, and in the second scenario, we multiplied an R-MAT matrix with the restriction operator of order 8 on the right as described in section 5.3.2.

Trilinos ran out of memory when multiplying R-MAT matrices of scale larger than 21, or when using more than 256 processors. Figure 5.13a shows SpGEMM timings for up to 256 processors on scale 21 data. Sparse SUMMA is consistently faster than Trilinos's implementation, with the gap increasing with the processor count, reaching 66× on 256-way concurrency. Sparse SUMMA is also more memory efficient as Trilinos's matrix multiplication ran out of memory for $p = 1$ and $p = 4$ cores. The sweet spot for Trilinos seems to be around 120 cores, after which its performance degrades significantly.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

In the case of multiplying with the restriction operator, the speed and scalability of our implementation over EpetraExt is even more pronounced. This is shown in Figure 5.13b, where our code is 65X faster even on just 121 processors. Remarkably, our codes scales up to 4096 cores on this problem, as shown in section 5.3.2, while EpetraExt starts to slow down just beyond 16 cores. We also compared Sparse SUMMA with EpetraExt on matrices coming from physical problems, and the results for the full restriction operation ($\mathbf{SAS}^\mathsf{T}$) are shown in Figures 5.14.

In order to benchmark Trilinos's sparse matrix indexing capabilities, we used EpetraExt's permutation class that can permute row or columns of an Epetra_CrsMatrix by creating a map defined by the permutation, followed by an Epetra_Export operation to move data from the input object into the permuted object. We applied a random symmetric permutation on an R-MAT matrix, as done in section 5.1. Trilinos shows good scaling up to 121 cores, but then it starts slowing down as concurrency increases, eventually becoming over 10× slower than our SpRef implementation at 169 cores.

6. Conclusions and future work. We presented a flexible parallel sparse matrix-matrix multiplication (SpGEMM) algorithm, Sparse SUMMA, which scales to thousands of processors in distributed memory. We used Sparse SUMMA as a building block to design and implement scalable parallel routines for sparse matrix indexing (SpRef) and assignment (SpAsgn). These operations are important in the context of graph operations. They yield elegant algorithms for coarsening graphs by edge contraction as in Figure 6.1, extracting subgraphs, performing parallel breadth-first search from multiple source vertices, and performing batch updates to a graph.

We performed parallel complexity analyses of our primitives. In particular, using SpGEMM as a building block enabled the most general analysis of SpRef. Our extensive experiments confirmed that our implementation achieves the performance predicted by our analyses.

Our SpGEMM routine might be extended to handle matrix chain products. In particular, the sparse matrix triple product is used in the coarsening phase of algebraic multigrid [3]. Sparse matrix indexing and parallel graph contraction also require sparse matrix triple products [25]. Providing a first-class primitive for sparse matrix chain products would eliminate temporary intermediate products and allow more optimization, such as performing structure prediction [17] and determining the best order of multiplication based on the sparsity structure of the matrices.

As we show in section 5.3, our implementation spends more than 75% of its time in internode communication after 2000 processors. Scaling to higher concurrencies requires asymptotic reductions in communication volume. We are working on developing practical communication-avoiding algorithms [4] for sparse matrix-matrix multiplication (and consequently for sparse matrix indexing and assignment), which might require inventing efficient novel sparse data structures to support such algorithms.

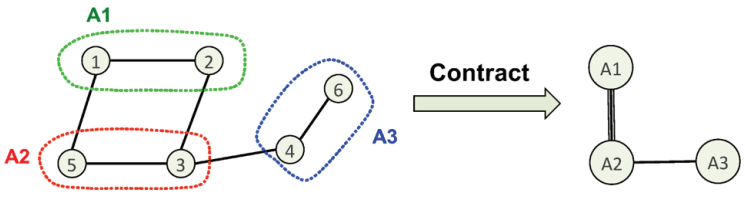


FIG. 6.1. *Example of graph coarsening using edge contraction, which can be implemented via a triple sparse matrix product* $\mathbf{SAS}^\mathsf{T}$ *where* $\mathbf{S}$ *is the restriction operator.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Our preliminary experiments suggest that synchronous algorithms for SpGEMM cause considerably higher load imbalance than asynchronous ones [9, section 7]. In particular, a truly one-sided implementation can perform up to 46% faster when multiplying two R-MAT matrices of scale 20 using 4000 processors. We will experiment with partitioned global address space (PGAS) languages, such as UPC [18], because the current implementations of one-sided MPI-2 were not able to deliver satisfactory performance when used to implement asynchronous versions of our algorithms.

As the number of cores per node increases due to multicore scaling, so does the contention on the network interface card. Without hierarchical parallelism that exploits the faster on-chip network, the flat MPI parallelism will be unscalable because more processes will be competing for the same network link. Therefore, designing hierarchically parallel SpGEMM and SpRef algorithms is an important future direction of our work.

REFERENCES

[1] Franklin, *Nersc's Cray XT4 System*, http://www.nersc.gov/users/computational-systems/franklin/.

[2] Combinatorial_Blas, *Combinatorial BLAS Library (MPI reference implementation)*, http://gauss.cs.ucsb.edu/ aydin/CombBLAS/html/index.html, 2012.

[3] M. Adams and J. W. Demmel, *Parallel multigrid solver for* 3D *unstructured finite element problems*, in Supercomputing '99: Proceedings of the 1999 ACM/IEEE Conference on Supercomputing, ACM, New York, IEEE, Washington, DC, 1999, p. 27.

[4] G. Ballard, J. Demmel, O. Holtz, and O. Schwartz, *Minimizing communication in numerical linear algebra*, SIAM. J. Matrix Anal. Appl, 32 (2011), pp. 866–901.

[5] W. L. Briggs, V. E. Henson, and S. F. McCormick, *A Multigrid Tutorial*, 2nd ed., SIAM, Philadelphia, 2000.

[6] A. Buluç and J. R. Gilbert, *New ideas in sparse matrix-matrix multiplication*, in Graph Algorithms in the Language of Linear Algebra, J. Kepner and J. Gilbert, eds., SIAM, Philadelphia, 2011, pp. 315–337.

[7] A. Buluç and J. R. Gilbert, *Challenges and advances in parallel sparse matrix-matrix multiplication*, in ICPP'08: Proc. of the International Conference on Parallel Processing, IEEE Computer Society, Portland, OR, 2008, pp. 503–510.

[8] A. Buluç and J. R. Gilbert, *On the representation and multiplication of hypersparse matrices*, in IPDPS'08: Proceedings of the 2008 IEEE International Symposium on Parallel and Distributed Processing, IEEE Computer Society, Washington, DC, 2008, pp. 1–11.

[9] A. Buluç and J. R. Gilbert, *Highly Parallel Sparse Matrix-Matrix Multiplication*, Technical report UCSB-CS-2010-10, Computer Science Department, University of California, Santa Barbara, CA, 2010.

[10] A. Buluç and J. R. Gilbert, *The combinatorial BLAS: Design, implementation, and applications*, Internat. J. High Performance Comput. Appl., 25 (2011), pp. 496–509.

[11] A. Buluç, J. R. Gilbert, and V. B. Shah, *Implementing sparse matrices for graph algorithms*, in Graph Algorithms in the Language of Linear Algebra, J. Kepner and J. Gilbert, eds., SIAM, Philadelphia, 2011, pp. 287–313.

[12] A. Buluç and K. Madduri, *Parallel breadth-first search on distributed memory systems*, in Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, ACM, New York, 2011, pp. 65:1–65:12.

[13] D. Chakrabarti, Y. Zhan, and C. Faloutsos, *R-MAT: A recursive model for graph mining*, in Proceedings of the 4th SIAM Conference on Data Mining, SIAM, Philadelphia, 2004, pp. 442–446.

[14] E. Chan, M. Heimlich, A. Purkayastha, and R. A. van de Geijn, *Collective communication: Theory, practice, and experience*, Concurrency Comput. Practice Experience, 19 (2007), pp. 1749–1783.

[15] C. Chevalier and I. Safro, *Comparison of coarsening schemes for multilevel graph partitioning*, in Learning and Intelligent Optimization: Third International Conference, LION 3. Selected Papers, Springer-Verlag, Berlin, Heidelberg, 2009, pp. 191–205.

[16] A. Chtchelkanova, J. Gunnels, G. Morrow, J. Overfelt, and R. A. van de Geijn, *Parallel implementation of BLAS: General techniques for Level* 3 *BLAS*, Concurrency Comput. Practice Experience, 9 (1997), pp. 837–857.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[17] E. Cohen, *Structure prediction and computation of sparse matrix products*, J. Combin. Optim., 2 (1998), pp. 307–332.
[18] UPC Consortium, *UPC Language Specifications,* v1.2, Technical report LBNL-59208, Lawrence Berkeley National Laboratory, Berkeley, CA, 2005.
[19] P. D'Alberto and A. Nicolau, *R-Kleene: A high-performance divide-and-conquer algorithm for the all-pair shortest path for densely connected networks*, Algorithmica, 47 (2007), pp. 203–213.
[20] T. A. Davis, *Direct Methods for Sparse Linear Systems*, SIAM, Philadelphia, 2006.
[21] T. A. Davis and Y. Hu, *The University of Florida sparse matrix collection*, ACM Trans. Math. Software, 38 (2011), p. 1.
[22] P. Erdös and A. Rényi, *On random graphs*, Publicationes Mathematicae, 6 (1959), pp. 290–297.
[23] R. A. van de Geijn and J. Watts, *SUMMA: Scalable universal matrix multiplication algorithm*, Concurrency Comput. Practice Experience, 9 (1997), pp. 255–274.
[24] J. R. Gilbert, C. Moler, and R. Schreiber, *Sparse matrices in MATLAB: Design and implementation*, SIAM J. Matrix Anal. Appl., 13 (1992), pp. 333–356.
[25] J. R. Gilbert, S. Reinhardt, and V. B. Shah, *A unified framework for numerical and combinatorial computing*, Comput. Sci. Engrg., 10 (2008), pp. 20–25.
[26] F. G. Gustavson, *Two fast algorithms for sparse matrices: Multiplication and permuted transposition*, ACM Trans. Math. Software, 4 (1978), pp. 250–269.
[27] B. Hendrickson and R. Leland, *A multilevel algorithm for partitioning graphs,* in Supercomputing '95: Proceedings of the 1995 ACM/IEEE Conference on Supercomputing, ACM, New York, IEEE, Washington, DC, 1995, p. 28.
[28] M. A. Heroux, R. A. Bartlett, V. E. Howle, R. J. Hoekstra, J. J. Hu, T. G. Kolda, R. B. Lehoucq, K. R. Long, R. P. Pawlowski, E. T. Phipps, A. G. Salinger, H. K. Thornquist, R. S. Tuminaro, J. M. Willenbring, A. Williams, and K. S. Stanley, *An overview of the Trilinos project*, ACM Trans. Math. Software, 31 (2005), pp. 397–423.
[29] H. Kaplan, M. Sharir, and E. Verbin, *Colored intersection searching via sparse rectangular matrix multiplication*, in Proceedings of the 22nd Annual Symposium on Computational Geometry, ACM, New York, 2006, pp. 52–60.
[30] J. W. H. Liu, *The multifrontal method for sparse matrix solution: Theory and practice*, SIAM Rev., 34 (1992), pp. 82–109.
[31] A. T. Ogielski and W. Aiello, *Sparse matrix computations on parallel processor arrays*, SIAM J. Sci. Comput., 14 (1993), pp. 519–530.
[32] G. Penn, *Efficient transitive closure of sparse matrices over closed semirings*, Theoret. Comput. Sci., 354 (2006), pp. 72–81.
[33] M. O. Rabin and V. V. Vazirani, *Maximum matchings in general graphs through randomization*, J. Algorithms, 10 (1989), pp. 557–567.
[34] V. B. Shah, *An Interactive System for Combinatorial Scientific Computing with an Emphasis on Programmer Productivity*, Ph.D. thesis, University of California, Santa Barbara, CA, 2007.
[35] S.-H. Teng, *Coarsening, sampling, and smoothing: Elements of the multilevel method*, in Parallel Processing, IMA Vol. Math. Appl., 105, Springer-Verlag, Berlin, 1999, pp. 247–276.
[36] S. Van Dongen, *Graph clustering via a discrete uncoupling process*, SIAM J. Matrix Anal. Appl., 30 (2008), pp. 121–141.
[37] I. Yamazaki and X. Li, *On techniques to improve robustness and scalability of a parallel hybrid linear solver*, in High Performance Computing for Computational Science, VECPAR, Springer-Verlag, Berlin, Heidelberg, 2010, pp. 421–434.
[38] R. Yuster and U. Zwick, *Detecting short directed cycles using rectangular matrix multiplication and dynamic programming*, in Proceedings of the 15th Annual ACM–SIAM Symposium on Discrete Algorithms, ACM, New York, SIAM, Philadelphia, 2004, pp. 254–260.

Downloaded 03/19/24 to 129.2.192.193 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.